

LAB 5: XÂY DỰNG DASHBOARD QUẢN LÝ NGƯỜI DÙNG (NEWS CMS)

Bối cảnh: Xây dựng phân hệ Admin cho một hệ thống Báo điện tử, giả sử hệ thống **chưa yêu cầu tính năng Đăng nhập**. Thay vào đó, sẽ tạo một menu thả xuống (Dropdown) để giả lập việc "Tôi đang đóng vai ai" nhằm kiểm tra tính năng phân quyền hiển thị.

Yêu cầu bài lab

1. Cấu trúc Dữ liệu giả lập (Mock Data)

Cần khởi tạo một State chứa danh sách người dùng ban đầu. Mỗi người dùng có cấu trúc như sau:

```
{
  id: "u1", // Bắt buộc phải có để làm Key
  name: "Nguyễn Văn A",
  email: "a@news.com",
  role: "Admin", // Các giá trị: SuperAdmin, Admin, Editor, Author, Contributor
  status: "Active" // Active hoặc Banned
}
```

2. Phân quyền Vai trò (Role-based Access)

Giao diện và các nút thao tác phải thay đổi linh hoạt (Conditional Rendering) dựa trên vai trò đang được giả lập.

Vai trò đang giả lập	Quyền xem danh sách	Nút "Thêm User"	Hành động trên từng dòng (Sửa/Xóa/Ban)
SuperAdmin	Toàn bộ	Hiển thị	Được thao tác trên tất cả mọi người
Admin	Toàn bộ	Hiển thị	Không được thao tác trên SuperAdmin và Admin khác
Editor	Toàn bộ	Ẩn	Chỉ được xem, ẩn toàn bộ nút thao tác
Author / Contributor	Không được xem	Ẩn	Hiển thị màn hình: " <i>Bạn không có quyền truy cập</i> "

3. Yêu cầu Chức năng (Functional Requirements)

1. **Bộ giả lập quyền (Role Simulator):** Đặt một thẻ `<select>` ở góc trên màn hình chứa 5 vai trò. Khi đổi vai trò ở đây, giao diện bên dưới phải cập nhật ngay lập tức theo bảng phân quyền ở trên.
2. **Hiển thị danh sách (Read):** Render danh sách người dùng dưới dạng bảng (Table) hoặc danh sách thẻ (Cards).
3. **Thêm người dùng (Create):** Có một form nhỏ gồm Tên, Email và Vai trò. Khi submit, thêm người dùng mới vào State (nhớ sinh ID ngẫu nhiên hoặc dùng `Date.now()`).
4. **Xóa/Ban người dùng (Delete/Update):** Mỗi dòng user có nút "Xóa" hoặc "Khóa tài khoản". Bấm vào sẽ cập nhật lại State danh sách.

4. Yêu cầu Kỹ thuật (Technical Constraints)

- **Bắt buộc dùng `map()` và `key`:** Phải sử dụng `user.id` làm key khi render danh sách. Tuyệt đối không dùng `index`.
- **Tối ưu Conditional Rendering:**
 - Sử dụng toán tử ba ngôi (`? :`) cho các trạng thái lớn (VD: Có quyền xem vs Màn hình cấm truy cập).
 - Sử dụng toán tử AND (`&&`) cho các thành phần nhỏ (VD: Nếu là SuperAdmin && hiện nút Xóa).
- **Props & State:** Chia nhỏ giao diện thành ít nhất 3 component: `RoleSimulator`, `UserForm`, và `UserList`. Truyền state và hàm xử lý (handler) từ Component cha xuống dưới dạng Props.

Hướng dẫn thực hiện

1. Cấu trúc Cây Component (Component Tree)

Hãy hướng dẫn sinh viên áp dụng nguyên tắc "**Lifting State Up**" (Đưa State lên trên). Toàn bộ dữ liệu quan trọng nhất sẽ được giữ ở Component cha trên cùng (App), sau đó truyền xuống các Component con qua Props.

App (Component Cha - Quản lý toàn bộ State)

- ├─ `RoleSimulator` (Dropdown chọn vai trò giả lập)
- ├─ `UserForm` (Form thêm người dùng mới)
- └─ `UserList` (Bảng hiển thị danh sách)

└─ UserItem (Từng dòng hiển thị thông tin 1 user)

2. Phân tích chi tiết từng Component

Gợi ý cho sinh viên xác định rõ Đầu vào (Props) và Trạng thái nội bộ (State) của từng phần.

A. Component App (Người quản lý tổng)

Đây là "trái tim" của ứng dụng, nơi chứa dữ liệu và các hàm xử lý logic.

- **State cần có:**
 - users: Mảng chứa danh sách người dùng.
 - currentRole: Chuỗi lưu vai trò đang giả lập (Mặc định: "SuperAdmin").
- **Hàm xử lý (Handlers):**
 - handleAddUser(newUser): Thêm user vào mảng users.
 - handleDeleteUser(id): Loại bỏ user có id tương ứng khỏi mảng.
 - handleChangeRole(role): Cập nhật state currentRole.
- **Render theo điều kiện mức độ Trang:** Nếu currentRole là "Author" hoặc "Contributor", App không render UserForm và UserList, mà chỉ render một thông báo "Bạn không có quyền truy cập".

B. Component RoleSimulator

Làm nhiệm vụ duy nhất là cho phép chọn vai trò đang giả lập.

- **Props nhận vào:**
 - currentRole: Để hiển thị giá trị đang được chọn trên <select>.
 - onChangeRole: Gọi hàm này (truyền lên Cha) mỗi khi người dùng đổi option.

C. Component UserForm

Khu vực nhập liệu để tạo người dùng mới.

- **Props nhận vào:**
 - onAdd: Hàm được gọi khi ấn nút "Submit", dùng để gửi dữ liệu form lên Cha.
- **State nội bộ:** Cần các State nhỏ gọn như nameInput, emailInput, roleSelect để lưu trữ dữ liệu người dùng đang gõ vào form (Controlled Components).
- **Render theo điều kiện:** Ẩn toàn bộ form này đi nếu currentRole của hệ thống đang là "Editor" (vì Editor không có quyền tạo).

D. Component `UserList` & `UserItem`

Khu vực render danh sách bằng `.map()`. Để code sạch sẽ, nên tách riêng từng hàng (row) ra thành `UserItem`.

- **UserList Props:** `users` (mảng dữ liệu), `currentRole`, `onDelete`.
- **UserItem Props:** `user` (dữ liệu của 1 người cụ thể), `currentRole`, `onDelete`.
- **Render theo điều kiện trong `UserItem` (Khúc khó nhất của sinh viên):**

Nút "Xóa" chỉ được render (&&) nếu thỏa mãn các logic phức tạp. Bạn có thể gợi ý sinh viên viết một hàm nhỏ bên trong `UserItem` tên là `canDelete()` trả về `true/false`.

- *Ví dụ logic `canDelete`:* Nếu `currentRole` là "SuperAdmin" thì luôn `true`. Nếu `currentRole` là "Admin" thì trả về `true` CHỈ KHI `user.role` không phải là "SuperAdmin" và "Admin".

3. Mã giả (*Code Skeleton*) gợi ý cấu trúc

Bạn có thể cung cấp đoạn mã khung này để sinh viên điền tiếp logic vào các chỗ trống ...:

```
function App() {
  const [users, setUsers] = useState(mockData);
  const [currentRole, setCurrentRole] = useState('SuperAdmin');

  // Các hàm xử lý logic
  const addUser = (newUser) => { ... }
  const deleteUser = (id) => { ... }

  // Render theo điều kiện cấp cao nhất
  const hasAccess = currentRole === 'SuperAdmin' || currentRole === 'Admin' ||
currentRole === 'Editor';

  return (
    <div className="dashboard">
      <h1>Hệ thống Quản lý Người dùng</h1>

      <RoleSimulator
        currentRole={currentRole}
        onChangeRole={setCurrentRole}
      />

      {/* Render theo điều kiện: Cấm Author/Contributor */}
      {!hasAccess ? (
        <h2>Bạn không có quyền truy cập trang này.</h2>
```

```
    ) : (  
      <>  
        { /* Editor không thấy Form thêm user */}  
        {currentRole !== 'Editor' && (  
          <UserForm onAdd={addUser} />  
        )}  
  
        <UserList  
          users={users}  
          currentRole={currentRole}  
          onDelete={deleteUser}  
        />  
      </>  
    )}  
  </div>  
);  
}
```